The Great Bitwise Bake Off

Ranam Hamoud, Yuning Yao and **Godwin Addetsi**

Abstract

This paper presents a system that generates novel dessert recipes using a multi-objective genetic algorithm. Drawing inspiration from evolutionary recipe generation approaches, our system evolves populations of recipes across 11 distinct dessert categories by balancing three key objectives: coherence with established culinary patterns, simplicity in the number of ingredients, and novelty of the resulting combinations. The approach uses a structured knowledge base of 260 real recipes, where ingredients are annotated with functional roles and semantic classes. A custom genetic algorithm, employing domain-specific crossover and mutation operators, explores this space, and category-specific templates uphold the plausibility of the generated cooking instructions. The system outputs 550 ranked recipes, demonstrating its capacity to create diverse and realistic desserts. We reflect on the system's performance through the lens of computational creativity and discuss the role of template-based instruction generation in balancing constraint satisfaction with creative exploration.

Introduction

Computational creativity in the domain of recipe generation presents a unique challenge: balancing culinary plausibility with novel ingredient combinations. Systems must navigate complex constraints, including ingredient functionality, cultural conventions, and ingredient interactions, and produce outputs that are both novel and valuable (with value proxied here by executability) (Jordanous 2016). The PIERRE system from (Morris et al. 2012) demonstrated the potential of neuro-symbolic approaches for this task. In this paper, we present a genetic algorithm-based approach to recipe generation that extends this line of research. Our system operates across multiple dessert categories, including cakes, cookies, pies, and tarts, each with distinct structural requirements and ingredient constraints. Unlike instruction generation systems, we use category-specific templates for cooking procedures and evolve only the ingredient combinations and their integration points into these procedures. Our design draws inspiration from EvoRecipes (Razzaq et al. 2023), which evolves context-aware recipes.

Our contributions include: (1) a multi-objective genetic algorithm that balances coherence, simplicity, and novelty; (2) a structured knowledge base incorporating ingredient roles and classes derived from real recipe data; (3) template-based instruction generation that preserves procedural plausibility; and (4) evaluation of 550 generated recipes across multiple dessert categories. We demonstrate how this approach can generate novel and realistic recipes that retain the culinary logic essential for executable results.

The paper proceeds as follows: Section 2 details our system architecture and the genetic algorithm implementation. Section 3 presents our reflections, development processes, evaluation methodology, and results. Section 4 discusses the system's creative capabilities and limitations, and Section 5 concludes with directions for future work.

System Description

Our system operates in three stages: (1) knowledge extraction from structured recipe data, computing ingredient statistics and building TF-IDF matrices per category; (2) genetic algorithm initialization with constraint-satisfying random recipes; and (3) iterative evolution through tournament selection, single-point crossover, four-type mutation, constraint-based repair, and elitist replacement over 50 generations.

Knowledge Base Construction

We constructed a structured knowledge base from 260 dessert recipes sourced from (Kaggle user: thedevastator). Using GPT-4o via the OpenRouter API, we transformed raw recipe data into a normalized JSON format. Each recipe includes structured ingredients with amount, unit, ingredient name, class (e.g., *fruit*, *grain*, *dairy*), and functional role (e.g., *fat*, *sweetener*, *structure*). This annotation enables semantic understanding of ingredient relationships and constraints. We computed statistical distributions (mean, standard deviation, min, max) for each ingredient-unit combination across the dataset to create realistic quantity generation during evolution.

Category Templates and Constraints

We defined 11 dessert categories (pie, cake, cookie, cobbler, crisp, tart, pudding, ice cream, brownie, bar, fudge), each with:

- **Must-have roles**: Essential functional ingredients (e.g., cakes require structure, fat, sweetener, moisture, flavoring).

- **Must-have classes**: Required ingredient categories.
- **Minimum ingredients**: Category-specific minima reflecting real dessert complexity.
- **Direction templates**: Fixed cooking instruction sequences that preserve the proper procedure.

Genetic Algorithm Implementation

The core of our system is a custom genetic algorithm that evolves recipe populations across 50 generations with a population size of 50 individuals per category. This evolutionary approach uses systematic exploration of the recipe space and sustains culinary plausibility through carefully designed genetic operators and constraint handling mechanisms.

Representation and Encoding Each recipe is represented as a variable-length genotype consisting of two primary components: an ingredient list and a step mapping. The ingredient list contains structured entries with normalized amounts, appropriate units, and semantic annotations including functional roles (e.g., *fat*, *sweetener*, *structure*) and ingredient classes (e.g., *fruit*, *grain*, *dairy*). The step mapping associates each ingredient with specific instruction steps from category-specific templates, ensuring that ingredients are incorporated at logically appropriate stages of the cooking process (prep, mix, and assemble phases). During crossover, step mappings are inherited from the parent recipe that contributed each ingredient. Initial recipes are created by first satisfying all must-have roles and classes, then randomly assigning ingredients to template steps tagged with appropriate phases. This representation allows for meaningful genetic operations and maintains the structural integrity required for executable recipes.

Fitness Evaluation and Multi-Objective Optimization We evaluate recipes using three complementary objectives that balance culinary logic with creative innovation:

- **Coherence**: Measured using TF-IDF vectorization and cosine similarity to existing recipes in the category-specific knowledge base. For each category, we build a separate TF-IDF matrix from only the training recipes in that category, then calculate the maximum cosine similarity between the ingredient vector of a candidate recipe and all training vectors. Higher values indicate more conventional, proven combinations.
- **Simplicity**: Defined as the ingredient count normalized by a 50-ingredient maximum (lower is better). This favors practical, straightforward recipes that are easier for home cooks to execute, yet still permits complexity when justified by other objectives.
- **Novelty**: Calculated as 1 - coherence score, this objective explicitly encourages divergence from existing recipes in the training data. By directly opposing coherence, it creates a productive tension that drives the exploration of unconventional but potentially valuable ingredient combinations.

The overall fitness function combines these objectives using a weighted sum approach: $0.4 \times \text{coherence} - 0.3 \times \text{simplicity\_normalized} + 0.3 \times \text{novelty}$, where simplicity is normalized by dividing by 50 (maximum ingredients). This specific weighting reflects our prioritization of plausibility (coherence) while maintaining pressure for practicality (fewer ingredients preferred, hence negative weight) and innovation (novelty).

Genetic Operators and Evolutionary Mechanisms Our implementation has several domain-specific genetic operators designed to effectively navigate the recipe space:

- **Crossover**: We implement single-point crossover on ingredient lists with an 80% crossover rate, where parent recipes are split at a randomly selected point and their ingredient segments are exchanged. This operator enables successful ingredient combinations from different parents to be recombined into potentially superior offspring. The step mappings are inherited based on the provenance of each ingredient, preserving logical consistency in the cooking process.
- **Mutation**: Our mutation operators introduce controlled variation through four mechanisms applied probabilistically: amount modification with 20% probability (adjusting quantities using the learned statistical distributions), ingredient addition with 10% probability (selecting from the ingredient pool), ingredient removal with 10% probability (respecting minimum constraints), and step reassignment with 20% probability (changing incorporation timing). These rates balance exploration with stability.
- **Repair**: A domain-specific repair operator detects and corrects constraint violations by adding missing required ingredients when recipes fail to satisfy category-specific role or class requirements. This operator guarantees that all recipes meet basic culinary standards and still permits the genetic algorithm to explore intermediate states that may violate some constraints.
- **Selection**: We apply tournament selection with $k = 3$, which provides sufficient selection pressure and preserves population diversity. Also, we implement 10% elitism, preserving the best solutions from each generation to ensure monotonic improvement in fitness across generations.

Constraint Handling and Domain Knowledge Integration Our constraint handling system employs a dual approach using both hard and soft constraints to balance creativity with culinary plausibility:

- **Hard Constraints**: We enforce category-specific minimum ingredient counts that reflect real-world dessert complexity: fudge (2), bars and ice cream (3), cookies (3), brownies and crisps (4), and cakes and pies (5). Recipes violating these constraints trigger the repair operator, which adds missing ingredients that satisfy role or class requirements.
- **Soft Constraints**: We require recipes to include at least 75% of must-have roles and classes for their category, with violations resulting in fitness penalties rather than outright rejection. This approach supports the exploration

of unconventional recipes that may omit some traditional ingredients.

- **Statistical Plausibility**: Ingredient amounts are generated using normal distributions derived from real recipe data, with means and standard deviations calculated for each ingredient-unit combination. The generated quantities remain within realistic ranges observed in actual culinary practice.

The evolutionary process runs for 50 generations per category, with population diversity maintained through duplicate detection (comparing ingredient lists) and replacement with newly generated recipes. We found that this duration provides sufficient time for convergence and enables substantial exploration of the recipe space. The algorithm outputs 50 ranked recipes per category, providing a diverse set of solutions that represent different trade-offs between our three optimization objectives.

Instruction and Name Generation Cooking instructions are generated by copying category-specific direction templates and populating them with evolved ingredients. Each template contains sequential cooking steps with empty ingredient lists. During finalization, the system maps each ingredient to its assigned step (based on the evolved step mapping), inserting ingredients into the appropriate template positions to create complete, executable directions.

Recipe names are procedurally generated using pattern-based combinations of adjectives (e.g., 'Heavenly', 'Divine'), ingredient-specific descriptors (e.g., 'Chocolate', 'Lemon'), style prefixes (e.g., 'Grandma's', 'Chef's'), and category names. The system uses eight different name patterns, unique within each category through duplicate detection, producing diverse titles like 'Chocolate Divine Cake' or 'Country Pie Magic'.

Development Process and Reflections

Our development process centered on reconciling open-ended exploration with culinary constraints. Grounding search in ingredient roles and classes helped us avoid degenerate outputs while still enabling creative recombination.

Inspiring Set Selection and Knowledge Representation

We curated 260 dessert recipes to seed genetic operations with reliable patterns. Roles-and-classes constraints were used as flexible targets that can be partially met, reducing brittleness while preserving culinary intent.

Algorithm Design Choices

Building on the genetic algorithm baseline, we introduce multi-objective optimization to balance coherence, simplicity, and novelty. The 75% coverage threshold for roles and classes kept plausibility in view while leaving room for inventive departures from strict conventions.

Template-Based Instruction Generation

Template-based instruction generation creates a hybrid: evolutionary composition of ingredients paired with structured procedures. Procedural validity is retained, and ingredient innovation remains the target, mitigating weaknesses of purely evolutionary or purely rule-driven systems.

Parameter Calibration and Evolutionary Dynamics

We calibrated parameters (50 generations, population 50, tournament size 3, crossover 80%) through systematic small-sweep experiments. Fitness weights (coherence 0.4, simplicity -0.3, novelty 0.3), create a balance between culinary structure and exploratory breadth.

Reflections on Constraint Handling

A two-constraint scheme combines evolutionary flexibility with culinary knowledge. The repair operator acts as a safety net that corrects missing roles or classes, so exploration can traverse imperfect intermediates without moving away from plausible outcomes.

Reflections on Creativity Evaluation

Evaluating creativity in recipes requires both novelty and value (Jordanous 2016), where we use executability as a practical proxy for value in this domain. We combined quantitative metrics (TF-IDF similarity, constraint satisfaction) with qualitative inspection. We observed substantial divergence from the training set (for example, many top-ranked recipes have maximum cosine similarity < 0.5), though metrics cannot judge flavor or desirability. Ideally, chef and home-cook trials would complement computation; practical limits kept evaluation to computational and visual plausibility in this round. Human-in-the-loop studies are a natural next step.

Evaluation and Discussion

We evaluated our system by generating 550 recipes (50 per category) and analyzing them across multiple dimensions of computational creativity. Below we report grounded descriptive statistics over coherence, novelty, and ingredient counts, followed by category-level patterns and illustrative examples.

Output Analysis

Overall, generated outputs favor culinary plausibility while allowing room for creative variation. Across all 550 recipes, median coherence is 0.745, median novelty is 0.255, and median ingredient count is 7. These distributions indicate that the search tends toward familiar combinations but many candidates surfaced with lower similarity to training examples.

Creativity Assessment

We assess the system's performance across multiple dimensions of computational creativity:

- **Novelty**: Recipes show significant divergence from training data, with novel ingredient combinations (measured as maximum cosine similarity < 0.5) observed among many top-ranked outputs; novelty varies by category as reported below

- **Typicality**: Generated recipes maintain category-typical structure through template-based instructions
- **Quality**: Fitness scores indicate successful balance of coherence, simplicity, and novelty objectives
- **Value**: Recipes maintain executable cooking procedures with plausible ingredient amounts

Category-Level Patterns

Categories with tighter procedural and ingredient conventions (for example, ice cream, brownie, and tart) exhibit higher median coherence and lower novelty, consistent with constrained spaces: ice cream (median coherence 0.867, median novelty 0.133, median 6 ingredients), brownie (0.816, 0.184, 8), tart (0.803, 0.197, 6). More flexible categories such as cookie and bar display lower coherence and higher novelty: cookie (median coherence 0.601, novelty 0.399, median 8 ingredients), bar (0.632, 0.368, 7). Pie balances both (median coherence 0.625, novelty 0.375, median 7 ingredients), reflecting diverse fillings and crust variants. These patterns align with the system's optimization, where the weighted objective allows novelty to increase more in categories with broader combinatorial latitude.

At the tails, we observe categories occasionally hitting coherence near 1.0 (for example, top tart, ice cream, and fudge examples), reflecting ingredient lists that closely match training regularities. However, lower-coherence outliers appear more often in cookie and bar, where relaxed constraints permit unusual but still executable combinations.

Illustrative Examples

Representative top-ranked outputs include: pie ("Country Pie Magic") with 6 ingredients, coherence 0.970 and novelty 0.030; cookie ("Morning Cookie Joy") with 7 ingredients, coherence 0.744 and novelty 0.256; bar ("City Bar Melody") with 5 ingredients, coherence 0.681 and novelty 0.319. These examples show the trade-off across categories: tighter forms gravitate toward higher coherence, while more open forms allow more novelty without violating structural constraints.

Cookbook Presentation and Generative Enhancement

To present the results in a more accessible and emotionally engaging format, we created a digital cookbook titled Perfect Days: Sensational Breakfast Dessert Recipes. The collection features five generated desserts that reinterpret classic morning flavors such as brownies, cookies, pie, and pudding as sweet breakfast indulgences. These recipes were selected for clarity, simplicity, and creative balance, illustrating how computational generation can combine practicality with originality.

Each recipe includes a structured ingredient list, clear step-by-step directions generated from system templates, and an accompanying 'Behind the Flavor' narrative. These short descriptive texts, generated with ChatGPT, were refined to give each recipe a personal and story-like character. Rather than treating the results as purely computational outputs, the stories evoke emotion and memory. For example, Morning Brownie Passion describes the dessert as a reflection of the sunrise, turning breakfast flavors into comfort and warmth.

The visuals were created using OpenAI's DALL·E based on the ingredient combinations, colors, and textures of the recipes. All images follow a consistent style with soft morning light, natural tones, and a cozy, home-baked aesthetic that visually supports the cookbook's theme.

An introductory 'letter to the reader,' an ingredient-based index, and an acknowledgments section were also generated and refined using ChatGPT to emulate the tone and structure of professional cookbooks. Together, these components transform the raw algorithmic outputs into a coherent and human-centered creative artifact.

This stage demonstrates how generative AI can enhance the presentation of computational creativity projects by combining narrative, imagery, and design. The recipes themselves were generated by the genetic algorithm; their storytelling, visual style, and overall presentation were enriched through collaboration between human input and generative tools. This integration shows how different generative systems can work together to make algorithmic creativity more relatable and emotionally engaging.

Limitations and Challenges

Although template-based instruction generation ensures procedural coherence, it limits creativity in cooking methods. The system occasionally produces recipes with incompatible ingredient textures or unrealistic flavor profiles, highlighting the challenge of capturing implicit culinary knowledge. The reliance on statistical patterns from training data also constrains truly revolutionary combinations. Furthermore, the large language models used for ingredient and role classification can introduce misclassifications and inconsistencies, which may propagate into constraint checks and fitness evaluation.

Comparison with PIERRE

Unlike PIERRE's neuro-symbolic approach that generates both ingredients and instructions (Morris et al. 2012), our system separates these concerns. We evolve ingredient combinations and use fixed instruction templates, providing stronger guarantees of procedural plausibility but potentially limiting creative cooking methods. Related work such as Chef Watson (Varshney et al. 2019) explores flavor pairing and knowledge-driven composition, in contrast our multi-objective optimization explicitly balances the trade-off between coherence and novelty through an evolutionary search.

Conclusion and Future Work

We have presented a genetic algorithm-based system for generating novel dessert recipes that successfully balances culinary plausibility with creative exploration. By combining structured ingredient knowledge with category-specific templates and multi-objective optimization, our approach generates 550 diverse recipes across 11 dessert categories. The system demonstrates that explicit constraint handling

and template-based instruction generation can produce executable recipes and foster novel ingredient combinations.

For future work, we plan to incorporate more sophisticated culinary knowledge, including flavor pairing theories and chemical interaction models. Integrating neural language models for instruction generation could enhance creativity in cooking methods and maintain procedural coherence. In addition, user studies with professional chefs and home cooks would provide valuable real-world evaluation of the generated recipes' quality and creativity. Incorporating explainable AI techniques (Llano et al. 2020) could also help users understand why certain ingredient combinations were chosen, making the creative process more transparent.

The separation of ingredient evolution from instruction generation presents a promising framework for computational creativity systems in other domains where procedural knowledge and combinatorial creativity must be balanced. Our work contributes to the growing body of research demonstrating that constrained evolutionary approaches can produce valuable creative outputs in complex, knowledge-rich domains.

References

Jordanous, A. 2016. Four pppperspectives on computational creativity in theory and in practice. *Connection Science* 28(2):194–216.

Kaggle user: thedevastator. Better recipes for a better life. https://www.kaggle.com/datasets/thedevastator/better-recipes-for-a-better-life. Accessed: 2025-10.

Llano, M. T.; d'Inverno, M.; Yee-King, M.; McCormack, J.; Ilsar, A.; Pease, A.; and Colton, S. 2020. Explainable computational creativity. In *Proceedings of the 11th International Conference on Computational Creativity (ICCC'20)*, 334–341. Coimbra, Portugal: Association for Computational Creativity.

Morris, R. G.; Burton, S. H.; Bodily, P. M.; and Ventura, D. 2012. Soup over bean of pure joy: Culinary ruminations of an artificial chef. In *Proceedings of the Third International Conference on Computational Creativity (ICCC 2012)*, 119–125.

Razzaq, M. S.; Maqbool, F.; Ilyas, M.; and Jabeen, H. 2023. Evorecipes: A generative approach for evolving context-aware recipes. *IEEE Access* 11:74148–74164.

Varshney, L. R.; Pinel, F.; Varshney, K. R.; Bhattacharjya, D.; Schörgendorfer, A.; and Chee, Y.-M. 2019. A big data approach to computational creativity: The curious case of chef watson. *IBM Journal of Research and Development* 63(1):8618410.